2ª Lista de Exercícios de Linguagens Formais e Autômatos
2o Semestre de 2021
Profa. Gina Maira B. de Oliveira

Data de Envio: até 01/08/2022 às 14 H

* Exercícios marcados como EXTRA, não precisam ser enviados.

** Exercícios marcados como DELAY, podem ser enviados posteriormente (até 07/08), mas é aconselhado que sejam feitos antes da 2ª prova como preparação para a mesma

1) **Lema do Bombeamento para linguagens regulares**

   a. Prove que as linguagens a seguir não são regulares

   L1 = $\{w \in \{0,1\}^* \mid w = w_1 0 w_1^R\}$. Ex: 011**0**110.

   L2 = $\{w \in \{0,1\}^* \mid w = a^i b^j$, sendo $i \geq j, j \geq 0\}$ Ex: aaab.

   b. Livro Hopcroft, Ullmann e Motwani (Português)
      - Exercício 4.1.1 (letras d, f) (pág.138 e 139)
      - Exercício 4.1.2 (letras b) (pág.139)
      - EXTRA:Exercício 4.1.1 (letras c, e) (pág.138 e 139)
      - EXTRA: Exercício 4.1.2 (letras a, e) (pág.139)

2) **Simplificação e normalização de Gramáticas Livres de Contexto**
   a. Livro Hopcroft e Ullmann (Inglês)
      - Exercício 4.10 (pág.104)
      - Exercício 4.12 (pág.104) – Forma Normal de Chomsky
      - EXTRA Exercício 4.14 (pág.104) - (Greibach)

   b. Livro Hopcroft, Ullmann e Motwani (Português)
      - EXTRA Exercício 7.1.2 (pág.290)
      - EXTRA Exercício 7.1.4 (pág.290)
      - Exercício 7.1.3 (pág.290)

   c. Livro Paulo Blauth
      - Exercício 3.11 letra (b) (pág.129)

3) **Ambiguidade**

a. Complemente a gramática de expressões vista em aula incluindo as operações de potenciação (^) e subtração (-), e o operando z (mantendo x):
G = ({E},{+, -, *, ^, [,]},P,E), P={E → E+E | E*E | E-E | E^E | [E] | x | z)}
Remova a ambiguidade dessa gramática, considerando as precedências dos operadores aritméticos.
Mostre todas as árvores de derivação possíveis nas duas gramáticas (ambígua e não ambígua) para a expressão: x-[x+z*x]^z

b. Seja a gramática a seguir para expressões lógicas
G=({S}, {∧ , ¬ , → , (,) }, P, S) e P = {S → S∧S | S→S | ¬S | (S) | a | b | c)
Remova a ambiguidade considerando as seguintes precedências para os operadores lógicos: ¬ > → > ∧
Mostre todas as árvores de derivação possíveis nas duas gramáticas (ambígua e não ambígua) para a expressão: b→a∧(¬b∧c)

4) **Autômato de Pilha**

a. Livro Hopcroft e Ullmann (Inglês)
- Exercício 5.1 (pág.120): esse exercício se refere às linguagens (LLC) do exercício 4.1 (fazer 4.1(a), 4.1(c) e 4.1(f)
- EXTRA:Exercício 5.2 (pág.120))
- EXTRA: Exercício 5.7 (item a) (pág.121)
- EXTRA:Exercício 5.6 (pág.121)

b. Livro Paulo Blauth
- Exercício 3.3 (item f) (pág.129)
- EXTRA Exercício 3.3 (itens c e d) (pág.129). *Dica: a linguagem em c, na verdade, é regular. Então vc pode resolver utilizando um A.Pilha e ignorando as operações de pilha.

5) **Lema do Bombeamento para Linguagens Livres de Contexto**

a. Seja L = {w1 ϵ {a,b}* | w1 = ww}
Ex. L ={ξ, aa, bb, aaaa, abab, baba, ...}
Provar que L não é livre de contexto usando o L. Bombeamento para LLC.

b. Livro Hopcroft e Ullmann (Inglês)
- Exercício 6.1 (itens a, d) (pág.141)
- Exercício 6.2 (itens c) (pág.141)
- EXTRA: Exercício 6.1 (item c - difícil) (pág.141)
- EXTRA: Exercício 6.2 (item e- verificar a diferença para o exercício (a) acima) (pág.142)

c. Livro Paulo Blauth

- EXTRA: Exercício 3.14 (item b) (pág.129)

****** PARTE B ******

6) Remoção da Recursão à Esquerda

Seja a gramática G a seguir: G=({S,A,B,C},{a,b,c},P,S), onde:

P= {S-> Aa | B | C, A-> Aaa | Sa | BB | CC, B-> b, C-> c }

a) Descreva a linguagem gerada por G.
b) Transforme G para remover transições com recursão à esquerda.

7) Máquina de Turing

Livro Paulo Blauth

- Exercício 4.4 (item e) (pág.150)
- DELAY Exercício 4.4 (itens a, b, c) (pág.150)
- EXTRA: Exercício 4.4 (itens d) (pág.150)

8) DELAY Seja o problema da lista de substituições de letras vista em sala:

AT = EAT
A = ATE
LOW = LATER
PAN = PILLOW
ME = CARP

a) Mostre que CATERPILLAR = MAN
b) Mostre que não é possível obter EATER = MELOW

Livro do Paulo Blauth

Pag. 129

3.3 Desenvolva Autômatos com Pilha que reconheçam as seguintes linguagens:

a) $L_1 = \emptyset$

b) $L_2 = \{\varepsilon\}$

c) $L_3 = \{a, b\}^*$

d) $L_4 = \{w \mid w \text{ é palíndromo em } \{a, b\}^*\}$

e) $L_8 = \{w \mid w \text{ é Expressão Regular sobre o alfabeto } \{x\}\}$

f) $L_9 = \{ua^n va^n w \mid n \in \{1, 2\}, u, v, w \text{ são palavras de } \{a, b\}^* \text{ e } |u| = |v| = 5\}$

3.11 Considere a seguinte gramática:

$G = (\{S, X, Y, Z, A, B\}, \{a, b, u, v\}, P, S)$, onde:

$P = \{ S \rightarrow XYZ,$
$X \rightarrow AXA \mid BXB \mid Z \mid \varepsilon,$
$Y \rightarrow AYB \mid BYA \mid Z \mid \varepsilon,$
$A \rightarrow a, B \rightarrow b$
$Z \rightarrow Zu \mid Zv \mid \varepsilon\}$

a) Qual a linguagem gerada?

b) Simplifique a gramática.

3.14 Explique intuitivamente por que e prove que as seguintes linguagens não são Livres do Contexto:

a) $L_{10} = \{ww \mid w \text{ é palavra de } \{a, b\}^*\}$

b) $L_{11} = \{a^n b^n a^m \mid n \geq 0, m \geq 0 \text{ e } n \neq m\}$

Pag.150

4.4 Para cada uma das linguagens abaixo, desenvolva uma Máquina de Turing que a reconheça. Sugere-se que, pelo menos três sejam do tipo com Fita Limitada.

a) $L_1 = \{a^n b^n c^n \mid n \geq 0\}$

b) $L_2 = \{w \mid w$ possui o mesmo número de símbolos a, b e c$\}$

c) $L_3 = \{a^n b^m a^{n+m} \mid n \geq 0$ e $m \geq 0\}$

d) $L_4 = \{wcw \mid w$ é palavra de $\{a, b\}^*\}$

e) $L_5 = \{ww \mid w$ é palavra de $\{a, b\}^*\}$

f) $L_6 = \{(awwa)^n \mid w$ é palavra de $\{a, b\}^*$ e $n \geq 0\}$

g) $L_7 = \{w \mid w = a^1 b^2 a^3 b^4 \ldots a^{n-1} b^n$ e n é par$\}$

Livro Hopcroft, Ullmann e Motwani (Português)

- Pgs. 138 e 139

bemos que x e y consistem apenas em símbolos 0. O lema do bombeamento nos diz que xz está em L_{eq}, se L_{eq} é regular. Essa conclusão é o caso $k = 0$ no lema do bombeamento. ! Porém, xz tem n 1's, pois todos os 1's de w estão em z. Entretanto, xz também tem menos de n 0's, porque perdemos os 0's de y. Como $y \neq \varepsilon$, sabemos que não pode haver mais de que $n - 1$ 0's entre x e z. Assim, depois de supor que L_{eq} uma linguagem regular, provamos um fato conhecido como falso, de que xz está em L_{eq}. Temos uma prova por contradição de que L_{eq} não é regular. □

Exemplo 4.3: Vamos mostrar que a linguagem L_{pr} que consiste em todos os strings de 1's cujo comprimento é um número primo não é uma linguagem regular. Vamos supor que fosse. Então, haveria uma constante n satisfazendo às condições do lema do bombeamento. Considere algum primo $p \geq n + 2$; tem de haver tal p, pois existe uma infinidade de primos. Seja $w = 1^p$.

Pelo lema do bombeamento, podemos dividir $w = xyz$ de tal forma que $y \neq \varepsilon$ e $|xy| \leq n$. Seja $|y| = m$. Então, $|xz| = p - m$. Agora, considere o string $xy^{p-m}z$, que deve estar em L_{pr} pelo lema do bombeamento, se L_{pr} realmente é regular. Porém,

$$|xy^{p-m}z| = |xz| + (p-m)\,|y| = p - m + (p-m)\,m + (m+1)(p-m)$$

Parece que $xy^{p-m}z$ não é um primo, pois tem dois fatores $m + 1$ e $p - m$. Contudo, devemos verificar que nenhum desses fatores é 1, pois então $(m + 1)(p - m)$ poderia ser um primo, afinal de contas. No entanto, $m + 1 > 1$, porque $y \neq \varepsilon$ nos diz que $m \geq 1$. Além disso, $p - m > 1$, porque $p \geq n + 2$ foi escolhido, e $m \leq n$, pois

$$m = |y| \leq |xy| \leq n$$

Portanto, $p - m \geq 2$.

Mais uma vez, começamos supondo que a linguagem em questão era regular e derivamos uma contradição, mostrando que algum string não pertencente a linguagem era obrigado pelo lema do bombeamento a estar na linguagem. Desse modo, concluímos que L_{pr} não é uma linguagem regular. □

4.1.3 Exercícios para a Seção 4.1

Exercício 4.1.1: Prove que as linguagens a seguir não são regulares.

a) $\{0^n 1^n \mid n \geq 1\}$. Essa linguagem, consistindo em um string de 0's seguido por um string de igual comprimento de 1's, é a linguagem L_{01} que consideramos informalmente no início da seção. Aqui, você deve aplicar o lema do bombeamento à prova.

[1]Observe no texto a seguir que também poderíamos ter sucesso escolhendo $k = 2$, ou na verdade qualquer valor de k diferente de 1.

b) O conjunto de strings de parênteses balanceados. Esses são os strings de caracteres "(" e ")" que podem aparecer em uma expressão aritmética bem formada.

* c) $\{0^n 1 0^n \mid n \geq 1\}$.

d) $\{0^n 1^{n^2} \mid n \geq 1\}$! n e m são inteiros arbitrários!. -

e) $\{0^n 1^m \mid n \leq m\}$. -

f) $\{0^n 1^{2n} \mid n \geq 1\}$. -

! Exercício 4.1.2: Prove que as linguagens a seguir não são regulares.

* a) $\{0^n \mid n$ é um quadrado perfeito$\}$.

- b) $\{0^n \mid n$ é um cubo perfeito$\}$.

- c) $\{0^n \mid n$ é uma potência de 2$\}$.

- d) O conjunto de strings de 0's e 1's cujo comprimento é um quadrado perfeito.

e) O conjunto de strings de 0's e 1's que têm a forma ww, isto é, algum string repetido.

- f) O conjunto de strings de 0's e 1's que têm a forma ww^R, isto é, algum string seguido por seu reverso. (Consulte na Seção 4.2.2 uma definição formal do reverso de um string.)

g) O conjunto de strings de 0's e 1's da forma $w\bar{w}$, onde $\bar{w}$ é formado a partir de w, substituindo-se todos os símbolos 0 por símbolos 1 e vice-versa; por exemplo, $\bar{011} = 100$ e 01110 é um exemplo de um string na linguagem.

h) O conjunto de strings da forma $w1^n$, onde w é um string de 0's e 1's de comprimento n.

!! Exercício 4.1.3: Prove que as linguagens a seguir não são regulares.

a) O conjunto de strings de 0's e 1's, começando com 1, tal que, quando interpretado como um inteiro, esse inteiro é um número primo.

b) O conjunto de strings da forma $0^i 1^j$ tais que o máximo divisor comum de i e j é 1.

! Exercício 4.1.4: Quando tentamos aplicar o lema do bombeamento a uma linguagem regular, o "adversário ganha", e não podemos completar a prova. Mostre o que acontece de errado quando escolhemos L como uma das seguintes linguagens:

Pag. 290

do uma produção que introduza um terminal como o primeiro símbolo do corpo e que seja seguida por variáveis para gerar todas as seqüências de variáveis que poderiam ter sido geradas no processo de geração desse terminal.

Essa forma, chamada *Forma Normal de Greibach*, em homenagem a Sheila Greibach, que foi, a primeira a apresentar um modo de construir tais gramáticas, tem várias conseqüências interessantes. Tendo em vista que cada uso de uma produção introduz exatamente um terminal em uma forma sentencial, um string de comprimento n tem uma derivação de exatamente n etapas. Além disso, se aplicarmos a construção de PDA do Teorema 6.13 a uma gramática na Forma Normal de Greibach, obteremos um PDA sem ε-regras, mostrando assim que é sempre possível eliminar tais transições de um PDA.

* **Exercício 7.1.2:** Comece com a gramática:

$$\begin{aligned} S &\rightarrow ASB \mid \varepsilon \\ A &\rightarrow aAS \mid a \\ B &\rightarrow SbS \mid A \mid bb \end{aligned}$$

a) Elimine as ε-produções.

b) Elimine quaisquer produções unitárias na gramática resultante.

c) Elimine quaisquer símbolos inúteis na gramática resultante.

d) Coloque a gramática resultante na forma normal de Chomsky.

Exercício 7.1.3: Repita o Exercício 7.1.2 para a seguinte gramática:

$$\begin{aligned} S &\rightarrow 0A0 \mid 1B1 \mid BB \\ A &\rightarrow C \\ B &\rightarrow S \mid A \\ C &\rightarrow S \mid \varepsilon \end{aligned}$$

Exercício 7.1.4: Repita o Exercício 7.1.2 para a seguinte gramática:

$$\begin{aligned} S &\rightarrow AAA \mid B \\ A &\rightarrow aA \mid B \\ B &\rightarrow \varepsilon \end{aligned}$$

The pair of productions $S \to \alpha$ and $A \to \alpha_1 B \alpha_2$ will receive a number if α contains a variable in C_{ad}, A is in C_{ad}, and B is in C_{bc}. If this pair is assigned the number i, then define N_i to be the set of n such that for some m,

$$S \Rightarrow \alpha \overset{*}{\Rightarrow} x_1 A x_2 \Rightarrow x_1 \alpha_1 B \alpha_2 x_2 \overset{*}{\Rightarrow} a^n b^m c^m d^n.$$

Also define M_i to be the set of m such that for some n,

$$S \Rightarrow \alpha \overset{*}{\Rightarrow} x_1 A x_2 \Rightarrow x_1 \alpha_1 B \alpha_2 x_2 \overset{*}{\Rightarrow} a^n b^m c^m d^n.$$

Once again, for any n in N_i and m in M_i,

$$S \overset{*}{\underset{G_2}{\Rightarrow}} a^n b^m c^m d^n,$$

and thus it follows from Lemma 4.5 that G_2 generates all but a finite number of sentences in $\{a^n b^n c^n d^n \mid n \geq 1\}$. We conclude that for some n, $a^n b^n c^n d^n$ is in both $L(G_1)$ and $L(G_2)$. This sentence has two leftmost derivations in G. □

EXERCISES

4.1 Give context-free grammars generating the following sets.

S a) The set of palindromes (strings that read the same forward as backward) over alphabet $\{a, b\}$.

b) The set of all strings of balanced parentheses, i.e., each left parenthesis has a matching right parenthesis and pairs of matching parentheses are properly nested.

* c) The set of all strings over alphabet $\{a, b\}$ with exactly twice as many a's as b's.

d) The set of all strings over alphabet $\{a, b, \cdot, +, *, (,), \epsilon, \emptyset\}$ that are well-formed regular expressions over alphabet $\{a, b\}$. Note that we must distinguish between ϵ as the empty string and as a symbol in a regular expression. We use ϵ in the latter case.

* e) The set of all strings over alphabet $\{a, b\}$ not of the form ww for some string w.

f) $\{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$.

* **4.2** Let G be the grammar

$$S \to aS \mid aSbS \mid \epsilon.$$

Prove that

$$L(G) = \{x \mid \text{each prefix of } x \text{ has at least as many } a\text{'s as } b\text{'s}\}.$$

* **4.3** For $i \geq 1$, let b_i denote the string in $\mathbf{1(0 + 1)^*}$ that is the binary representation of i. Construct a CFG generating

$$\{0, 1, \#\}^+ - \{b_1 \# b_2 \# \cdots \# b_n \mid n \geq 1\}.$$

* **4.4** Construct a CFG generating the set

$$\{w \# w^R \# \mid w \text{ in } \mathbf{(0 + 1)^+}\}^*.$$

* **4.5** The grammar

$$E \to E + E \mid E * E \mid (E) \mid \mathbf{id}$$

generates the set of arithmetic expressions with $+$, $*$, parentheses and **id**. The grammar is ambiguous since **id** + **id** * **id** can be generated by two distinct leftmost derivations.

a) Construct an equivalent unambiguous grammar.

b) Construct an unambiguous grammar for all arithmetic expressions with no redundant parentheses. A set of parentheses is redundant if its removal does not change the expression, e.g., the parentheses are redundant in **id** + (**id** * **id**) but not in (**id** + **id**) * **id**.

* **4.6** Suppose G is a CFG with m variables and no right side of a production longer than ℓ. Show that if $A \overset{*}{\underset{G}{\Rightarrow}} \epsilon$, then there is a derivation of no more than $\frac{\ell^m - 1}{\ell - 1}$ steps by which A derives ϵ. How close to this bound can you actually come?

* **4.7** Show that for each CFG G there is a constant c such that if w is in $L(G)$, and $w \neq \epsilon$, then w has a derivation of no more than $c|w|$ steps.

4.8 Let G be the grammar

$$S \to aB \mid bA$$
$$A \to a \mid aS \mid bAA$$
$$B \to b \mid bS \mid aBB$$

For the string *aaabbabbba* find a

a) leftmost derivation, b) rightmost derivation, c) parse tree.

* **4.9** Is the grammar in Exercise 4.8 unambiguous?

4.10 Find a CFG with no useless symbols equivalent to

$$S \to AB \mid CA \qquad B \to BC \mid AB$$
$$A \to a \qquad C \to aB \mid b$$

4.11 Suppose G is a CFG and w, of length ℓ, is in $L(G)$. How long is a derivation of w in G if

a) G is in CNF b) G is in GNF.

4.12 Let G be the CFG generating well-formed formulas of propositional calculus with predicates p and q:

$$S \to \sim S \mid [S \supset S] \mid p \mid q.$$

The terminals are p, q, $\sim$, [,], and $\supset$. Find a Chomsky normal-form grammar generating $L(G)$.

4.13 Show that conversion to Chomsky normal form can square the number of productions in a grammar. [*Hint*: Consider the removal of unit productions.]

4.14 Find a Greibach normal-form grammar equivalent to the following CFG:

$$S \to AA \mid 0$$
$$A \to SS \mid 1$$

4.15 Show that every CFL without ϵ can be generated by a CFG all of whose productions are of the form $A \to a$, or $A \to BC$, where $B \neq C$ and if $A \to \alpha_1 B \alpha_2$ and $A \to \gamma_1 B \gamma_2$ are productions, then $\alpha_1 = \gamma_1 = \epsilon$ or $\alpha_2 = \gamma_2 = \epsilon$.

*S **4.16** Show that every CFL without ϵ is generated by a CFG all of whose productions are of the form $A \to a$, $A \to aB$, and $A \to aBC$.

Material: Livro Hopcroft e Ullmann (Inglês)
Pags. 103 e 104

Pag. 120

These are also required by $\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$. The productions for the remaining variables and the relevant moves of the PDA are:

1) $[q_0, X, q_0] \to 0[q_0, X, q_0][q_0, X, q_0]$
   $[q_0, X, q_0] \to 0[q_0, X, q_1][q_1, X, q_0]$
   $[q_0, X, q_1] \to 0[q_0, X, q_0][q_0, X, q_1]$
   $[q_0, X, q_1] \to 0[q_0, X, q_1][q_1, X, q_1]$
   since $\delta(q_0, 0, X) = \{(q_0, XX)\}$.
2) $[q_0, X, q_1] \to 1$ since $\delta(q_0, 1, X) = \{(q_1, \epsilon)\}$.
3) $[q_1, Z_0, q_1] \to \epsilon$ since $\delta(q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$.
4) $[q_1, X, q_1] \to \epsilon$ since $\delta(q_1, \epsilon, X) = \{(q_1, \epsilon)\}$.
5) $[q_1, X, q_1] \to 1$ since $\delta(q_1, 1, X) = \{(q_1, \epsilon)\}$.

REMOVE USELESS SYMBOLS

It should be noted that there are no productions for the variables $[q_1, X, q_0]$ and $[q_1, Z_0, q_0]$. As all productions for $[q_0, X, q_0]$ and $[q_0, Z_0, q_0]$ have $[q_1, X, q_0]$ or $[q_1, Z_0, q_0]$ on the right, no terminal string can be derived from $[q_0, X, q_0]$ or $[q_0, Z_0, q_0]$ either. Deleting all productions involving one of these four variables on either the right or left, we end up with the following productions.

$$S \to [q_0, Z_0, q_1], \qquad [q_1, Z_0, q_1] \to \epsilon,$$
$$[q_0, Z_0, q_1] \to 0[q_0, X, q_1][q_1, Z_0, q_1], \qquad [q_1, X, q_1] \to \epsilon,$$
$$[q_1, X, q_1] \to 0[q_0, X, q_1][q_1, X, q_1], \qquad [q_1, X, q_1] \to 1.$$
$$[q_0, X, q_1] \to 1,$$

We summarize Theorems 5.1 through 5.4 as follows. The three statements below are equivalent:

1) L is a context-free language.
2) $L = N(M_1)$ for some PDA M_1.
3) $L = L(M_2)$ for some PDA M_2.

EXERCISES

5.1 Construct pushdown automata for each of the languages in Exercise 4.1.

5.2 Construct a PDA equivalent to the following grammar.

$$S \to aAA, \qquad A \to aS \mid bS \mid a.$$

5.3 Complete the proof of Theorem 5.3 by showing that every CFL L is the set accepted by some PDA even if ϵ is in L. [*Hint:* Add a second state to the PDA for $L - \{\epsilon\}$.]

5.4 Show that if L is a CFL, then there is a PDA M accepting L by final state such that M has at most two states and makes no ϵ-moves.

Pag. 121

* 5.5

a) Show that if L is a CFL, then L is $L(M)$ for some PDA M such that if $\delta(q, a, X)$ contains (p, γ), then $|\gamma| \leq 2$.

b) Show that M of part (a) can be further restricted so that if $\delta(q, a, X)$ contains (p, γ), then γ is either ϵ (a pop move), X (no change to the stack), or YX for some stack symbol Y (a push move).

c) Can we put a bound on the number of states of M in part (a) and still have a PDA for any CFL?

d) Can we put a bound on the number of states in part (b)?

5.6 Give a grammar for the language $N(M)$ where

$$M = (\{q_0, q_1\}, \{0, 1\}, \{Z_0, X\}, \delta, q_0, Z_0, \emptyset)$$

and δ is given by

$$\delta(q_0, 1, Z_0) = \{(q_0, XZ_0)\}, \qquad \delta(q_0, \epsilon, Z_0) = \{(q_0, \epsilon)\},$$
$$\delta(q_0, 1, X) = \{(q_0, XX)\}, \qquad \delta(q_1, 1, X) = \{(q_1, \epsilon)\},$$
$$\delta(q_0, 0, X) = \{(q_1, X)\}, \qquad \delta(q_1, 0, Z_0) = \{(q_0, Z_0)\}.$$

5.7 The deterministic PDA (DPDA) is not equivalent to the nondeterministic PDA. For example, the language

$$L = \{0^n1^n \mid n \geq 1\} \cup \{0^n1^{2n} \mid n \geq 1\}$$

is a CFL that is not accepted by any DPDA.

a) Show that L is a CFL.

** b) Prove that L is not accepted by a DPDA.

5.8 A language L is said to have the *prefix property* if no word in L is a proper prefix of another word in L. Show that if L is $N(M)$ for DPDA M, then L has the prefix property. Is the foregoing necessarily true if L is $N(M)$ for a nondeterministic PDA M?

* 5.9 Show that L is $N(M)$ for some DPDA M if and only if L is $L(M')$ for some DPDA M', and L has the prefix property.

5.10 A *two-way PDA* (2PDA) is a PDA that is permitted to move either way on its input. Like the two-way FA, it accepts by moving off the right end of its input in a final state. Show that $L = \{0^n1^n2^n \mid n \geq 1\}$ is accepted by a 2PDA. We shall show in the next chapter that L is not a CFL, by the way, so 2PDA's are not equivalent to PDA's.

*S 5.11 Write a program to translate a regular expression to a finite automaton.

* 5.12 The grammar

$$E \to E + E \mid E * E \mid (E) \mid \mathbf{id} \tag{5.5}$$

generates the set of arithmetic expressions with $+$, $*$, parentheses and id in *infix* notation (operator between the operands). The grammar

$$P \to +PP \mid *PP \mid \mathbf{id}$$

generates the set of arithmetic expressions in *prefix* notation (operator precedes the operands). Construct a program to translate arithmetic expressions from infix to prefix notation

Similarly, $V_{21} = V_{31} = V_{51} = \{A, C\}$, since only A and C have productions with a on the right.

To compute V_{ij} for $j > 1$, we must execute the for-loop of steps (6) and (7). We must match V_{ik} against $V_{i+k,j-k}$ for $k = 1, 2, \ldots, j-1$, seeking variable D in V_{ik} and E in $V_{i+k,j-k}$ such that DE is the right side of one or more productions. The left sides of these productions are adjoined to V_{ij}. The pattern in the table which corresponds to visiting V_{ik} and $V_{i+k,j-k}$ for $k = 1, 2, \ldots, j-1$ in turn is to simultaneously move down column i and up the diagonal extending from V_{ij} to the right, as shown in Fig. 6.10.

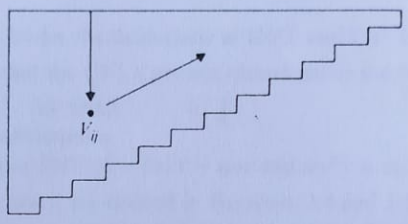


Fig. 6.10 Traversal pattern for computation of V_{ij}.

For example, let us compute V_{24}, assuming that the top three rows of Fig. 6.9 are filled in. We begin by looking at $V_{21} = \{A, C\}$ and $V_{33} = \{B\}$. The possible right-hand sides in $V_{21}V_{33}$ are AB and CB. Only the first of these is actually a right side, and it is a right side of two productions $S \to AB$ and $C \to AB$. Hence we add S and C to V_{24}. Next we consider $V_{22}V_{42} = \{B\}\{S, A\} = \{BS, BA\}$. Only BA is a right side, so we add the corresponding left side A to V_{24}. Finally, we consider $V_{23}V_{51} = \{B\}\{A, C\} = \{BA, BC\}$. BA and BC are each right sides, with left sides A and S, respectively. These are already in V_{24}, so we have $V_{24} = \{S, A, C\}$. Since S is a member of V_{15}, the string *baaba* is in the language generated by the grammar.

EXERCISES

6.1 Show that the following are not context-free languages.

a) $\{a^i b^j c^k \mid i < j < k\}$

b) $\{a^i b^j \mid j = i^2\}$

c) $\{a^i \mid i \text{ is a prime}\}$

d) the set of strings of a's, b's, and c's with an equal number of each

e) $\{a^n b^n c^m \mid n \leq m \leq 2n\}$

6.2* Which of the following are CFL's?

a) $\{a^i b^j \mid i \neq j \text{ and } i \neq 2j\}$

b) $(\mathbf{a} + \mathbf{b})^* - \{(a^n b^n)^n \mid n \geq 1\}$

c) $\{ww^R w \mid w \text{ is in } (\mathbf{a} + \mathbf{b})^*\}$

d) $\{b_i \# b_{i+1} \mid b_i \text{ is } i \text{ in binary}, i \geq 1\}$

Pag. 142

c) $\{wxw \mid w \text{ and } x \text{ are in } (\mathbf{a} + \mathbf{b})^*\}$
f) $(\mathbf{a} + \mathbf{b})^* - \{(a^n b)^n \mid n \geq 1\}$

6.3 Prove that the following are not CFL's.

a) $\{a^i b^j a^k \mid j = \max\{i, k\}\}$
b) $\{a^n b^n c^i \mid i \neq n\}$

[*Hint:* Use Ogden's lemma on a string of the form $a^n b^n c^{n!}$.]

6.4 Show that the CFL's are closed under the following operations:

* a) Quotient with a regular set, that is, if L is a CFL and R a regular set, then L/R is a CFL.
b) INIT
S** c) CYCLE
d) reversal

See Exercise 3.4 for the definitions of INIT and CYCLE.

* **6.5** Show that the CFL's are not closed under the following operations.

S a) MIN b) MAX c) $\frac{1}{2}$
d) Inverse substitution
e) INV, where $\text{INV}(L) = \{x \mid x = wyz \text{ and } wy^R z \text{ is in } L\}$

MIN, MAX, and $\frac{1}{2}$ are defined in Exercises 3.4 and 3.16.

* **6.6** Let Σ be an alphabet. Define homomorphisms h, h_1, and h_2 by $h(a) = h(\bar{a}) = a$, $h_1(a) = a$, $h_1(\bar{a}) = \epsilon$, $h_2(a) = \epsilon$, and $h_2(\bar{a}) = a$ for each a in Σ^*. For $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Sigma^*$, define

$$\text{Shuffle}(L_1, L_2) = \{x \mid \text{for some } y \text{ in } h^{-1}(x),\ h_1(y) \text{ is in } L_1 \text{ and } h_2(y) \text{ is in } L_2\}.$$

That is, the Shuffle of L_1 and L_2 is the set of words formed by "shuffling" a word of L_1 with a word of L_2. Symbols from the two words need not alternate as in a "perfect shuffle."

a) Show that the Shuffle of two regular sets is regular.
b) Prove that the Shuffle of two CFL's is not necessarily a CFL.
c) Prove that the Shuffle of a CFL and a regular set is a CFL.

* **6.7** A *Dyck Language* is a language with k types of balanced parentheses. Formally, each Dyck language is, for some k, $L(G_k)$, where G_k is the grammar

$$S \to SS \mid [_1 S]_1 \mid [_2 S]_2 \mid \cdots \mid [_k S]_k \mid \epsilon.$$

For example, $[_1[_2[_1]_1[_2]_2]_2]_1$ is in the Dyck language with two kinds of parentheses. Prove that every CFL L is $h(L_D \cap R)$, where h is a homomorphism, R a regular set, and L_D a Dyck language. [*Hint:* Let L be accepted by empty stack by a PDA in the normal form of Exercise 5.5(b) where the moves only push or pop single symbols. Let the parentheses be $[_{abX}$ and $]_{abX}$, where $[_{abX}$ "means" on input a, stack symbol X is pushed, and matching parenthesis $]_{abX}$ "means" on input b, X may be popped (a or b may be ϵ). Then the Dyck language enforces the condition that the stack be handled consistently, i.e., if X is pushed, then it will still be X when it is popped. Let the regular set R enforce the condition that there be a sequence of states for which the push and pop moves are legal for inputs a and b, respectively. Let $h([_{abX}) = a$ and $h(]_{abX}) = b$.]

* **6.8** Show that if L is a CFL over a one-symbol alphabet, then L is regular. [*Hint:* Let n be the pumping lemma constant for L and let $L \subseteq 0^*$. Show that for every word of length n or more, say 0^m, there are p and q no greater than n such that 0^{p+iq} is in L for all $i \geq 0$. Then